

TranAD Preprocessing Studies

*Using SPT Data to Investigate the
Impact of Preprocessing on TranAD Training*

Amy Bender, JJ Chen, Katie Harrington, Walter Hopkins, Wei Quan, Luke Vaughan
Argonne National Laboratory

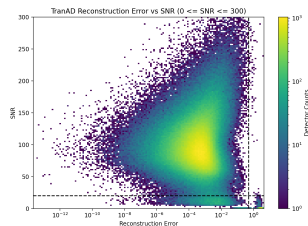
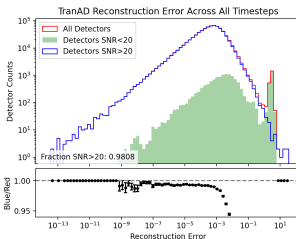
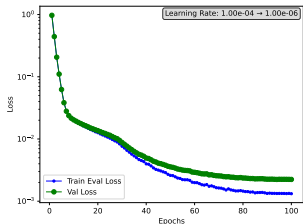
- **Preprocessing cuts** — define the three default selections applied to SPT data
- **Part 1: Initial study (with scaling)**
 - Default, No-Timestamp, and Loose preprocessing — results and a NaN crash
 - Diagnosis: a problematic scaling line identified as the cause of the crash
- **Part 2: Rerun without scaling**
 - Default, No-Timestamp, and Loose preprocessing repeated with scaling removed
 - All three trainings now complete successfully
- **Part 3: Configurable yaml pipeline**
 - Trim-config and No-Trim-config results using a new yaml-driven approach
 - Validates the configurable pipeline and highlights an open question on training time
- **Conclusions**

Preprocessing Selections Overview

- **Wafer cut**
 - Restrict to wafer w206, manually chosen as a “well-behaved” wafer
- **Unstable detector cut**
 - Compute each detector's median, 10th percentile, and 90th percentile over time
 - Keep only detectors with finite, positive median
 - Require 10th percentile $> 0.9 \times$ median
 - Require 90th percentile $< 1.1 \times$ median
 - In short: keep detectors within about $\pm 10\%$ of median most of the time
- **Bad timestamp cut**
 - For each kept detector, define valid values as finite and positive
 - Compute each detector's 1st percentile from valid samples
 - Keep timestamp only if every kept detector is finite, positive, and above its 1st percentile
 - In short: drop timestamps where any kept detector is invalid or unusually low

Since “data is data”, how does loosening preprocessing selections affect training of TranAD?

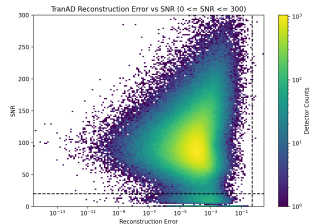
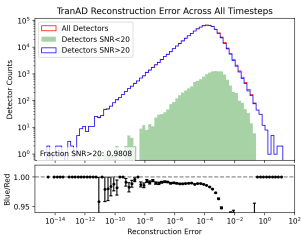
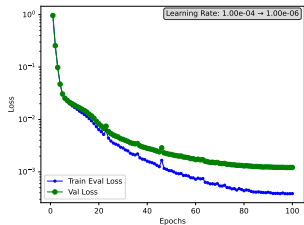
Default Results



- Wafer cut included: restrict to wafer w206
- Detector stability cut included
- Timestamp trim included for train/val only

- Train: (12268, 113, 1)
- Val: (3813, 113, 1)
- Test: (6428, 113, 1)
- Training time: 00:14:36

No-TimeStamp Results



- Wafer cut included: restrict to wafer w206
- Detector stability cut included
- No timestamp trim applied

- Train: (19274, 113, 1)
- Val: (6423, 113, 1)
- Test: (6428, 113, 1)
- Training time: 00:22:07

Loose Results

Epoch 17: train 0.4642, train_eval 0.4043, val 0.4507

Epoch 18: train 0.4562, train_eval 0.3989, val 0.4464

Epoch 19: train 0.4516, train_eval 0.3954, val 0.4442

Error at epoch 20: non-finite training loss (nan).

- No wafer cut applied
- No detector stability cut applied
- No timestamp trim applied
- Train: (19274, 2700, 1)
- Val: (6423, 2700, 1)
- Test: (6428, 2700, 1)
- Training time: unavailable

Scaling Change

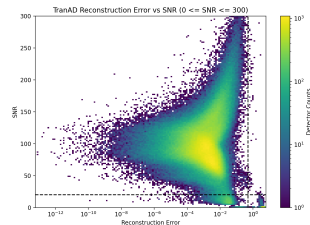
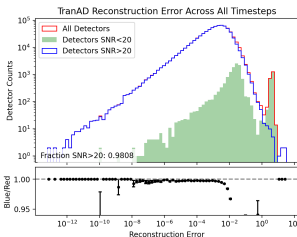
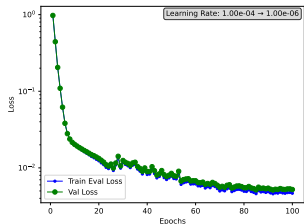
```
src/anldq/deep_learning/models/tranad.py

@@ -2,7 +2,6 @@
2 2  Implementations of the **TranAD** anomaly-detection model.
3 3  ***
4 4
5 - import math
6 5  from typing import Tuple
7 6
8 7  import torch

@@ -183,7 +182,7 @@ def _encode(self, src: Tensor, ctx: Tensor, tgt: Tensor):
183 182      tgt_rep = projected_target for decoder input
184 183      memory = encoder_memory for the decoder
185 184      ***
186 - merged = torch.cat((src, ctx), dim=-1) * math.sqrt(self.input_dims) # [B, T, 2*F]
185 + merged = torch.cat((src, ctx), dim=-1) # [B, T, 2*F]
187 186      projected = self.input_projection(merged) # [B, T, d_model]
188 187      encoded = self.pos_encoder(projected)
189 188      memory = self.transformer_encoder(encoded)
```

- Claude identified this line as the most likely issue behind the training failure.
- What happens if we remove this line and rerun all of the trainings?

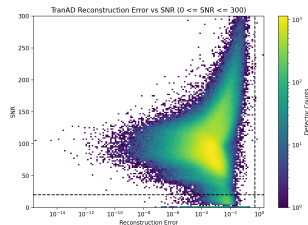
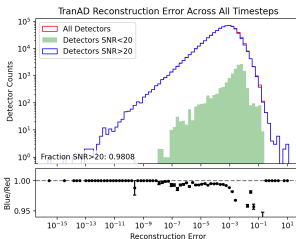
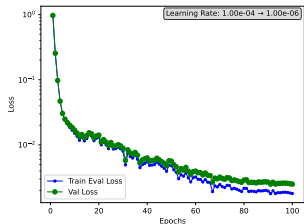
Default Results (rm scaling)



- Wafer cut included: restrict to wafer w206
- Detector stability cut included
- Timestamp trim included for train/val only

- Train: (12268, 113, 1)
- Val: (3813, 113, 1)
- Test: (6428, 113, 1)
- Training time: 00:14:52

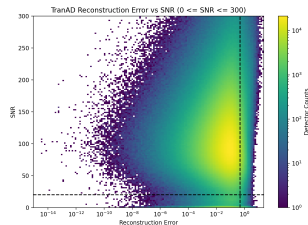
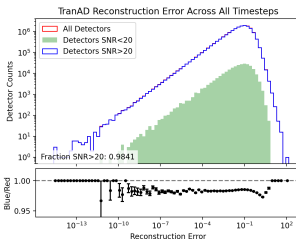
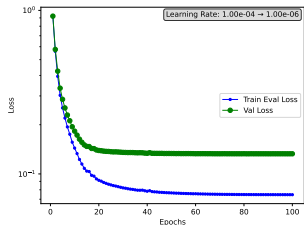
No-TimeStamp Results (rm scaling)



- Wafer cut included: restrict to wafer w206
- Detector stability cut included
- No timestamp trim applied

- Train: (19274, 113, 1)
- Val: (6423, 113, 1)
- Test: (6428, 113, 1)
- Training time: 00:22:46

Loose Results (rm scaling)



- No wafer cut applied
- No detector stability cut applied
- No timestamp trim applied

- Train: (19274, 2700, 1)
- Val: (6423, 2700, 1)
- Test: (6428, 2700, 1)
- Training time: 04:12:24

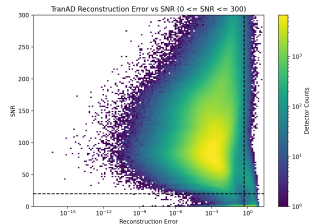
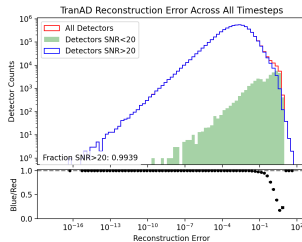
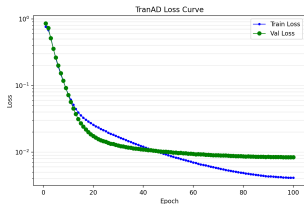
Pipeline Configuration

```
1 # SPT benchmark preprocessing with train-derived quality selection.
2 # Loader root is configured here. Output roots default to $ANL_QG_OUTPUT_DIR.
3
4 loader:
5   type: spt_benchmark
6   params:
7     root: /lcr/project/SPT3G/users/ac.welquan/ml_dqn/calibrator_responses
8     train_years: [2019, 2020, 2021]
9     test_years: [2022, 2023]
10    data_variant: smr
11    response_template: calibrator_responses_095ghz_(year).hdf5
12    smr_template: calibrator_responses_smr_095ghz_(year).hdf5
13    observation_id_key: Observation ID
14    require_monotonic_timestamps: true
15    load_response_reference: true
16    feature_names:
17      response: response
18      smr: SMR
19
20 steps:
21   - name: interpolate_snr_per_channel
22     - name: select_stable_channels
23     params:
24       low_quantile: 20.0
25       high_quantile: 80.0
26       tolerance: 0.05
27     reference_metadata_key: quality_reference_response
28   - name: filter_quality_timestamps
29     params:
30       require_monotonic: true
31       low_quantile: 1.0
32       high_quantile: 99.0
33       min_valid_fraction: 0.05
34
35 output:
36   save: true
37   root: null
38   experiment: spt
39   data_tag: true
40   file_name: train_processed.npz
```

```
1 # SPT benchmark preprocessing without quality trimming.
2 # Loader root is configured here. Output roots default to $ANL_QG_OUTPUT_DIR.
3
4 loader:
5   type: spt_benchmark
6   params:
7     root: /lcr/project/SPT3G/users/ac.welquan/ml_dqn/calibrator_responses
8     train_years: [2019, 2020, 2021]
9     test_years: [2022, 2023]
10    data_variant: smr
11    response_template: calibrator_responses_095ghz_(year).hdf5
12    smr_template: calibrator_responses_smr_095ghz_(year).hdf5
13    observation_id_key: Observation ID
14    require_monotonic_timestamps: true
15    load_response_reference: false
16    feature_names:
17      response: response
18      smr: SMR
19
20 steps:
21   - name: interpolate_snr_per_channel
22
23 output:
24   save: true
25   root: null
26   experiment: spt
27   data_tag: no_trim
28   file_name: train_processed.npz
```

- Taking inspiration from Joaquin, we can configure the pipeline via yaml.
- Left config: preprocessing cuts are enabled, including timestamp trimming and detector trimming.
- Right config: no trimming is applied; only NaNs are interpolated.

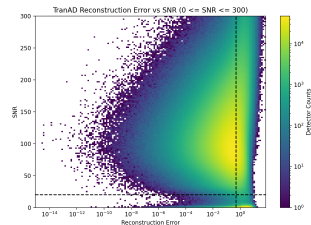
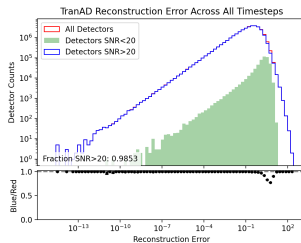
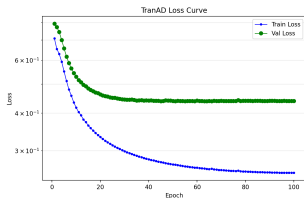
Trim Config Results



- Detector trimming enabled
- Timestamp trimming enabled
- Train/test split by years

- Train: (7976, 489, 1)
- Val: (2652, 489, 1)
- Test: (12610, 489, 1)
- Training time: 00:03:40

No-Trim Config Results



- No detector trimming applied
- No timestamp trimming applied
- Train/test split by years

- Train: (11694, 2700, 1)
- Val: (3892, 2700, 1)
- Test: (12610, 2700, 1)
- Training time: 00:06:48

- **Timestamp cuts act as an oracle.** Removing bad data sharpens the reconstruction-error peak. Without them, the signal is washed out.
- **Loose preprocessing with scaling enabled crashes.** This is a scaling instability, not a physics effect.
- **Removing scaling shifts the loss landscape** and visibly changes the 2D error histogram.
- **The yaml pipeline reproduces consistent qualitative behavior,** validating the configurable approach.
- **Open question:** the yaml no-trim config trains significantly faster despite 2700 channels. Cause is still under investigation.